Experiment No -07 (Group B)

Title:-

Student Record Management System using Linked List

Problem statement:-

Create a Student Record Management System using linked list

- Use a singly/doubly linked list to store student data (Roll No, Name, Marks).
- Perform operations: Add, Delete, Update, Search, and Sort.
- Display records in ascending/descending order based on marks or roll number.

Pre-requisite:-

- Basics of Python programming.
- Understanding of **classes** and variables.
- Knowledge of dynamic memory allocation.
- Basic understanding of **singly or doubly linked lists**.
- Familiarity with basic data-structure operations such as insertion, deletion, traversal, and searching.
- Functions and loops in Python.

Objective:-

The objective of this project is to develop a Student Record Management System using a linked list. The system stores student information such as Roll Number, Name, and Marks in dynamically allocated nodes.

Software and Hardware requirements:-

1. **Operating system:** Linux- Ubuntu 16.04 to 17.10, or Windows 7 to 10,
2. **RAM-** 2GB RAM (4GB preferable)
3. IDE :- Python 3
4. **Hardware**: Standard PC with keyboard input

Outcome:-

After completing this project, the student will be able to:

- Implement a singly/doubly linked list for storing records.
- Create and manipulate linked-list nodes dynamically.
- Perform insertion, deletion, updating, and searching operations.
- Apply sorting techniques to linked-list data.
- Display student records according to different sorting criteria.

Theory:-

Basic concept of linked list:-

- A linked list is a set of nodes where each node has two fields ' data' and 'link'.
- The 'data' field stores actual piece of information and 'link' field is used to point to next node.
- Basically 'link' field is nothing but address only.



Fig –Structure of Node

- Hence link list of integers 10, 20, 30, 40 is



Note that the 'link' field of last node consists of NULL which indicates end of list.

Representation of Linked List using Python

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Operations Implemented:

1. **Add Student:** Insert a record at the end.
2. **Display Records:** Traverse and print all student records.
3. **Delete Record:** Remove a record by Roll No.
4. **Update Record:** Modify student details by Roll No.
5. **Search Record:** Find a student by Roll No.
6. **Sort Records:** Sort records by Roll No or Marks (ascending/descending).

Algorithm:-

1. Add Student:

1. Create a new node with Roll No, Name, Marks.
2. If list is empty → set head to new node.
3. Else traverse to last node → attach new node at end.

2. Delete Student (by Roll No):

1. Traverse list until Roll No matches.
2. Adjust previous and next links to bypass node.
3. Delete node.

3. Update Student:

1. Traverse list to find Roll No.
2. If found, update Name/Marks.

4. Search Student:

1. Traverse list sequentially.
2. If Roll No matches → display details.

5. Sort Records:

1. Apply bubble sort on linked list nodes.
2. Swap Roll No/Marks accordingly.

Conclusion:

- A **Student Record Management System** was successfully implemented using a **Doubly Linked List**.
- Operations like **Add, Delete, Update, Search, Sort, and Display** were implemented.
- The practical helped in understanding **dynamic memory management** and **real-life applications of linked list**.

Viva Questions:-

1. What is the difference between **singly** and **doubly** linked lists?
2. Why a linked list is preferred over arrays in dynamic memory applications?
3. Which sorting technique was used in this program?
4. How can we optimize the search operation?
5. What is the time complexity of insert, delete, and search in linked lists?
6. What is a linked list?
7. What is a node?
8. What is a singly linked list?
9. What is a doubly linked list?
10. What are the advantages of using a linked list for this project?